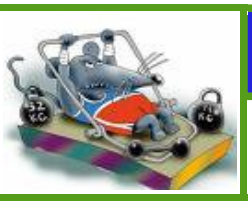
¡NO TE DOY DEBERES.  **TE DOY PODERES!**

Ahora me toca a mi. UXMLXP/XS-1

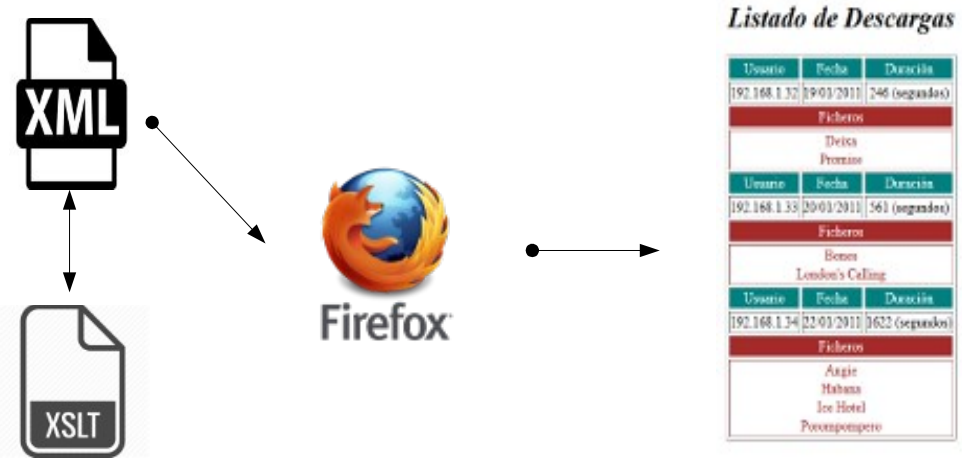
Llegados a este punto, la batería de ejercicios se transforma en una práctica guiada en la que de manera progresiva se van introduciendo los conceptos y sentencias de código necesarias para llevar a cabo transformaciones en un documento XML.

Partiendo del documento XML denominado **DocBaseXML.xml**, vamos a conocer la funcionalidad del elemento **<xsl:template match>**. Guarda esta hoja de estilo con el nombre *Ejemplo1.xsl* y asóciala en el XML Copy Editor (XML → XSL Transformación...), una vez se haya añadido al documento XML <?xml-stylesheet type="text/xsl" href="Ejemplo1.xsl"?> para que surta efecto. Eso sí, en el momento de activar la transformación, este editor genera una nueva pestaña con el código HTML resultante que posteriormente será renderizado por el navegador.

```
<?xml version='1.0' encoding="ISO-8859-1"?>
<xsl:stylesheet version="1.0"
xmlns:xsl="http://www.w3.org/1999/XSL/Transform">

<xsl:template match="/">
 <HTML>
   <HEAD><TITLE>Ejemplo1.xsl</TITLE></HEAD>
   <BODY>
    <CENTER>
     <H2>Patrón ajustado con el <U>nodo Raíz</U></H2>
    </CENTER>
   </BODY>
 </HTML>
</xsl:template>
</xsl:stylesheet>
```

Pero desde hace unas pocas versiones de los navegadores Firefox y Edge, se puede renderizar directamente un documento XML asociado ya a una hoja de estilo XSL. Sólo es cuestión de hacer un "**Abrir con**" de dicho documento XML e indicando el navegador con el que se quiere renderizar la salida.



Ahora vamos a ver como enterarnos de una primera funcionalidad del elemento **<xsl:apply-templates>**, ya que queremos poder utilizar más patrones en la hoja de estilo. Esta la guardas como *Ejemplo2.xsl*.

```
<?xml version='1.0' encoding="ISO-8859-1"?>
<xsl:stylesheet version="1.0"
xmlns:xsl="http://www.w3.org/1999/XSL/Transform">

<xsl:template match="/">
 <HTML>
   <HEAD><TITLE>Ejemplo2.xsl</TITLE></HEAD>
   <BODY>
    <CENTER>
     <H2>Patrón ajustado con el <U>nodo Raíz</U></H2>
     <xsl:apply-templates/>
    </CENTER>
   </BODY>
 </HTML>
</xsl:template>

<xsl:template match="notas">
   <H3>Patrón ajustado con el <U>nodo Notas</U></H3>
</xsl:template>

</xsl:stylesheet>
```

Por si esto no nos convenció demasiado, una vez más y notemos como ahora salen muchas líneas con el patrón Alumno. Ahora a guardar como *Ejemplo3.xsl*.

```
<?xml version='1.0' encoding="ISO-8859-1"?>
<xsl:stylesheet version="1.0"
xmlns:xsl="http://www.w3.org/1999/XSL/Transform">

<xsl:template match="/">
 <HTML>
   <HEAD><TITLE>Ejemplo3.xsl</TITLE></HEAD>
   <BODY>
    <CENTER>
     <H2>Patrón ajustado con el <U>nodo Raíz</U></H2>
     <xsl:apply-templates />
    </CENTER>
   </BODY>
 </HTML>
</xsl:template>

<xsl:template match="notas">
   <H3>Patrón ajustado con el <U>nodo Notas</U></H3>
   <xsl:apply-templates />
</xsl:template>

<xsl:template match="alumno">
   <H4>Patrón ajustado con el <U>elemento Alumno</U></H4>
</xsl:template>

</xsl:stylesheet>
```

Vista esta utilización a modo de muñeca rusa, añadimos a la última plantilla el elemento **<xsl:value-of>**, que tiene la posibilidad de trabajar con el atributo **select** o no. En este caso lo utilizamos igualándolo a ".", con lo que le estamos diciendo al procesador XSL que muestre el contenido completo de cada nodo **alumno** seleccionado. Guardar como *Ejemplo4.xsl*.

Hay que destacar, llegados a este punto, que se ha utilizado el elemento **<apply-templates>** como un mero "paga el que viene luego" en cascada. Como se verá más adelante, el elemento **<apply-templates>** tiene más posibilidades a través de su atributo **select**.

Dicho esto, volvemos al resultado obtenido en pantalla y vemos que queda un poco apelotonado todo, ¿verdad?

```
<?xml version='1.0' encoding="ISO-8859-1"?>
<xsl:stylesheet version="1.0"
xmlns:xsl="http://www.w3.org/1999/XSL/Transform">

<xsl:template match="/">
 <HTML>
   <HEAD><TITLE>Ejemplo4.xsl</TITLE></HEAD>
   <BODY>
    <CENTER>
     <H2>Patrón ajustado con el <U>nodo Raíz</U></H2>
     <xsl:apply-templates />
    </CENTER>
   </BODY>
 </HTML>
</xsl:template>

<xsl:template match="notas">
   <H3>Patrón ajustado con el <U>nodo Notas</U></H3>
   <xsl:apply-templates />
</xsl:template>

<xsl:template match="alumno">
   <H4>Patrón ajustado con el <U>elemento Alumno</U></H4>
   <xsl:value-of  select="."/>
</xsl:template>

</xsl:stylesheet>
```

Por eso ahora vamos a coger algunos elementos de cada nodo seleccionado y los pondremos entre etiquetas <p> y decorado con guiones y comas, de manera que la salida quede más mona. Lo importante es que las instrucciones <xsl:value-of> ahora muestran información concreta de elementos terminales. Este se guardará como *Ejemplo5.xsl*.

Detalle curioso a observar es que en el elemento <xsl:template match="alumno">, también valdría poner <xsl:template match="notas/alumno"> e incluso funcionaría <xsl:template match="./alumno">. Se puede probar en el simulador de patrones con el documento XML denominado **xml2.xml**.

```
<?xml version='1.0' encoding="ISO-8859-1"?>
<xsl:stylesheet version="1.0"
xmlns:xsl="http://www.w3.org/1999/XSL/Transform">

<xsl:template match="/">
 <HTML>
   <HEAD><TITLE>Ejemplo5.xsl</TITLE></HEAD>
   <BODY>
    <CENTER>
     <H2>Listado de Notas</H2>
     <HR></HR>
     <xsl:apply-templates select="notas/alumno"/>
     <HR></HR>
    </CENTER>
   </BODY>
 </HTML>
</xsl:template>

<xsl:template match="alumno">
   <P><B>Nombre y Apellidos: </B>
     <xsl:value-of select="nombre" />,
     <xsl:value-of select="apellidos" /></P>
   <P><B>Notas: </B>
     <xsl:value-of select="nota-teoria" /> ---
     <xsl:value-of select="nota-practica" /> ---
     <B><xsl:value-of select="nota-final" /></B></P>
</xsl:template>

</xsl:stylesheet>
```

Aunque se ve que puede funcionar lo anterior, lo correcto para evitar problemas posteriores, es que el patrón XPath que se consigna en la sentencia <xsl:template match=""> coincida con el de la sentencia <xsl:apply-templates select=""/>, como aquí:

Y aprovechamos este código para señalar la segunda funcionalidad del <xsl:apply-templates select=""/>: recuperar información de un nodo que contiene ocurrencias múltiples de algún dato.

El siguiente ejemplo del sitio W3Schools muestra claramente la utilización de esta herramienta para efectuar una diferenciación de color (aunque puedes probar a cambiar alguno de ellos de tamaño) de los elementos de un determinado nodo.

```
<?xml version='1.0' encoding="ISO-8859-1"?>
<xsl:stylesheet version="1.0"
xmlns:xsl="http://www.w3.org/1999/XSL/Transform">

<xsl:template match="/">
 <HTML>
  <HEAD>
    <TITLE>HojaFact1.xsl</TITLE>
  </HEAD>
  <BODY>
   <CENTER><H1>Listado de Facturas</H1></CENTER>
    <TABLE BORDER="1" ALIGN="CENTER">
     <xsl:apply-templates select="facturacion/factura">
       <xsl:sort select='codfact' order='ascending'/>
      </xsl:apply-templates>
    </TABLE>
  </BODY>
 </HTML>
</xsl:template>

<xsl:template match="facturacion/factura">
   <TR ALIGN="CENTER" BGCOLOR="YELLOW">
    <TD><B>Codigo Factura</B></TD>
    <TD><B>Estado</B></TD>
    <TD><B>Total</B></TD>
   </TR>
   <TR ALIGN="CENTER">
    <TD><xsl:value-of select="codfact"/></TD>
    <TD><xsl:value-of select="@estado"/></TD>
    <TD><xsl:value-of select="importe"/></TD>
   </TR>
   <TR ALIGN="CENTER" BGCOLOR="YELLOW">
     <TD COLSPAN="4"><B>Detalle de productos</B></TD>
    </TR>
    <TR ALIGN="CENTER" BGCOLOR="CYAN">
     <TD COLSPAN="4"><xsl:apply-templates select="detalle"/></TD>
    </TR>
</xsl:template>

<xsl:template match="detalle">
   <xsl:value-of select="producto"/><BR/>
</xsl:template>

</xsl:stylesheet>
```

Ahora avanzando un paso más, y como en la canción de los elefantes, nos traemos un nuevo amigo llamado <xsl:for-each>. Este como su colega <xsl:value-of> también tiene el atributo **select**, que sirve para lo mismo. Esto suele quedar muy bien para salidas tabulares, como en el siguiente ejemplo donde se va a colocar la información de cada nodo **alumno** seleccionado en fila de una tabla. Guardar como *Ejemplo6.xsl*.

Se aprovecha dentro de este mismo ejemplo (*Ejemplo6.xsl*), para ampliar horizontes del elemento <xsl:for-each select=""/> de forma que la salida de los datos se haga de forma ordenada. En el ejemplo, se ordena por ***nota-final***, utilizando la etiqueta, en este caso en modalidad de autocierre <xsl:sort> ya que los datos vienen ordenados de origen por apellidos. Ojo a los atributos select y order.

La plantilla se aplica de una tacada partiendo del nodo raíz, por lo que no se aplica **<xsl:apply-templates>.**

```
<?xml version='1.0' encoding="ISO-8859-1"?>
<xsl:stylesheet version="1.0" xmlns:xsl="http://www.w3.org /
1999/XSL/Transform">
 <xsl:template match="/">
  <HTML>
   <HEAD><TITLE>Ejemplo6.xsl</TITLE></HEAD>
   <BODY>
    <CENTER><H1>Listado de Notas</H1></CENTER>
    <TABLE BORDER="1" ALIGN="CENTER">
     <TR ALIGN="CENTER" BGCOLOR="CYAN">
      <TD COLSPAN="3">Alumno</TD>
      <TD COLSPAN="3">NOTAS</TD>
    </TR>
     <TR ALIGN="CENTER" BGCOLOR="YELLOW">
      <TD>Nombre</TD><TD>Apellidos</TD>
      <TD>Matricula</TD><TD>Teoria</TD>
      <TD>Practica</TD><TD>Final</TD>
     </TR>
     <xsl:for-each select="notas/alumno">
         <xsl:sort select="nota-final" order="ascending"/>
      <TR ALIGN="CENTER">
       <TD><xsl:value-of select="nombre"/></TD>
       <TD><xsl:value-of select="apellidos"/></TD>
       <TD><xsl:value-of select="nexped"/></TD>
       <TD><xsl:value-of select="nota-teoria"/></TD>
       <TD><xsl:value-of select="nota-practica"/></TD>
       <TD><B><xsl:value-of select="nota-final"/></B></TD>
      </TR>
     </xsl:for-each>
    </TABLE>
   </BODY>
  </HTML>
 </xsl:template>
</xsl:stylesheet>
```

Y fueron a buscar otro elefante, <xsl:if>. Este nos permite comprobar que un nodo cumple o no una serie de condiciones. La condición se articula a través del atributo **test** y sigue las mismas reglas establecidas al principio de este documento.

```
<?xml version='1.0' encoding="ISO-8859-1"?>
<xsl:stylesheet version="1.0"
xmlns:xsl="http://www.w3.org/1999/XSL/Transform">
<!—Lo que viene aquí, igual que en el anterior  -->
      <xsl:for-each select="notas/alumno">
        <xsl:sort select="nota-final" order="ascending"/>
       <xsl:if test="@convocatoria='Jun'">
        <TR ALIGN="CENTER">
         <TD><xsl:value-of select="nombre"/></TD>
         <TD><xsl:value-of select="apellidos"/></TD>
         <TD><xsl:value-of select="nexped"/></TD>
         <TD><xsl:value-of select="nota-teoria"/></TD>
         <TD><xsl:value-of select="nota-practica"/></TD>
         <TD><B><xsl:value-of select="nota-final"/></B></TD>
        </TR>
       </xsl:if>
      </xsl:for-each>
      </TABLE>
     </BODY>
    </HTML>
  </xsl:template>
</xsl:stylesheet>
```

Como se ve el resultado era el esperado. Pero y si en vez de lo anteriormente expuesto, hubiéramos optado por :

```
<?xml version='1.0' encoding="ISO-8859-1"?>
<xsl:stylesheet version="1.0"
xmlns:xsl="http://www.w3.org/1999/XSL/Transform">
<!—Lo que viene aquí, igual que en el anterior  -->
      <xsl:for-each select="notas/alumno[@convocatoria='Jun']">
        <xsl:sort select="nota-final" order="ascending"/>
         <TR ALIGN="CENTER">
           <TD><xsl:value-of select="nombre"/></TD>
           <TD><xsl:value-of select="apellidos"/></TD>
           <TD><xsl:value-of select="nexped"/></TD>
           <TD><xsl:value-of select="nota-teoria"/></TD>
           <TD><xsl:value-of select="nota-practica"/></TD>
           <TD><B><xsl:value-of select="nota-final"/></B></TD>
         </TR>
      </xsl:for-each>
      </TABLE>
     </BODY>
   </HTML>
  </xsl:template>
</xsl:stylesheet>
```

¿Sorpresa? Es simplemente poner en práctica lo visto. Otro elefante, que aunque no se balanceaba, es grandote porque funciona como un tres en uno. <xsl:choose> va en combinación con <xsl:when> y <xsl:otherwise> aporta una posibilidad parecida a la de un switch del lenguaje C. Este trozo de *Ejemplo7.xsl* nos muestra su funcionalidad:

```
<!—Lo del Ejemplo6 antes del for-each →

 <xsl:for-each select="notas/alumno">
   <TR ALIGN="CENTER">
     <TD><xsl:value-of select="nombre"/></TD>
     <TD><xsl:value-of select="apellidos"/></TD>
     <TD><xsl:value-of select="nexped"/></TD>
     <TD><xsl:value-of select="nota-teoria"/></TD>
     <TD><xsl:value-of select="nota-practica"/></TD>
     <TD><B>
       <xsl:choose>
         <xsl:when test="nota-final > '9.0'">Sobresaliente</xsl:when>
         <xsl:when test="nota-final > '7.0'">Notable</xsl:when>
         <xsl:otherwise>Aprobado</xsl:otherwise>
       </xsl:choose>
     </B></TD>
   </TR>

<!—Lo del Ejemplo6 después del for-each -->
```

Se consigue mostrar la tabla pero ahora en vez de aparecer la nota final en modo numérico se sustituye por Sobresaliente, Notable o Aprobado en función del valor de la nota final.

Para aquellos que no gusten de mezclar el código XSL y el HTML, no tengan cuidado. Porque pensando un poco el HTML que puede aparecer en una hoja de estilo, no son más que marcas de elementos y atributos de esos elementos. Pues todo tiene remedio excepto lo que todos sabemos.

<xsl:element> y <xsl:attribute> se encargan del apaño, como se muestra en el siguiente código *Ejemplo8.xsl*:

```
<?xml version='1.0' encoding="ISO-8859-1"?>
<xsl:stylesheet version="1.0" xmlns:xsl="http://www.w3.org/1999/XSL/Transform">

<xsl:template match="/">
  <xsl:element name="HTML">
    <xsl:element name="HEAD">
      <xsl:element name="TITLE">Ejemplo8.xsl</xsl:element>
    </xsl:element>
    <xsl:element name="BODY">
     <xsl:element name="CENTER">
      <xsl:element name="H1">Listado de Notas</xsl:element>
      </xsl:element>
      <xsl:element name="TABLE">
       <xsl:attribute name="BORDER">1</xsl:attribute>
       <xsl:attribute name="ALIGN">CENTER</xsl:attribute>
        <xsl:element name="TR">
         <xsl:attribute name="BGCOLOR">CYAN</xsl:attribute>
         <xsl:attribute name="ALIGN">CENTER</xsl:attribute>
          <xsl:element name="TD"><xsl:attribute name="COLSPAN">3</xsl:attribute>
            Alumno
          </xsl:element>
          <xsl:element name="TD"><xsl:attribute name="COLSPAN">3</xsl:attribute>
            NOTAS
          </xsl:element>
        </xsl:element>
       <xsl:element name="TR">
        <xsl:attribute name="BGCOLOR">YELLOW</xsl:attribute>
        <xsl:attribute name="ALIGN">CENTER</xsl:attribute>
          <xsl:element name="TD">Nombre</xsl:element>
          <xsl:element name="TD">Apellidos</xsl:element>
          <xsl:element name="TD">Matricula</xsl:element>
          <xsl:element name="TD">Teoria</xsl:element>
          <xsl:element name="TD">Practica</xsl:element>
          <xsl:element name="TD">Final</xsl:element>
       </xsl:element>
       <xsl:for-each select="notas/alumno">
        <xsl:element name="TR"><xsl:attribute name="ALIGN">CENTER</xsl:attribute>
          <xsl:element name="TD"><xsl:value-of select="nombre"/></xsl:element>
          <xsl:element name="TD"><xsl:value-of select="apellidos"/></xsl:element>
          <xsl:element name="TD"><xsl:value-of select="nexped"/></xsl:element>
          <xsl:element name="TD"><xsl:value-of select="nota-teoria"/></xsl:element>
          <xsl:element name="TD"><xsl:value-of select="nota-practica"/></xsl:element>
          <xsl:element name="TD">
            <xsl:element name="B">
             <xsl:copy>
               <xsl:choose>
                 <xsl:when test="nota-final >= '9.0'">Sobresaliente</xsl:when>
                 <xsl:when test="nota-final >= '7.0'">Notable</xsl:when>
                 <xsl:otherwise>Aprobado</xsl:otherwise>
               </xsl:choose>
             </xsl:copy>
             </xsl:element>
           </xsl:element>
        </xsl:element>
       </xsl:for-each>
    </xsl:element>
   </xsl:element>
  </xsl:element>
 </xsl:template>
</xsl:stylesheet>
```

Como se ve, los elementos (HTML,HEAD, BODY, CENTER, etc...) se envuelven en marcas **<xsl:element** name=”elemento HTML”> y los atributos en marcas **<xsl:attribute** name=”atributo HTML”>.

Si se quiere reproducir algún comentario tipo HTML para que aparezca en el documento HTML resultante, la cosa es igual de sencilla porque en vez de poner :

```
<!—Este es el comentario en HTML -->
```

en XSL a través de **<xsl:comment>** con ponerlo así vale :

```
<xsl:comment>Este es el mismo comentario anterior</xsl:comment>
```

Una versión más sofisticada de este elemento se muestra en el siguiente ejemplo XSL, *Ejemplo9.xsl*:

```
<?xml version="1.0" encoding="ISO-8859-1" ?>
<xsl:stylesheet version="1.0" xmlns:xsl="http://www.w3.org/1999/XSL/Transform">
  <xsl:template match="/">
    <HTML>
      <HEAD><TITLE>Ejemplo9.xsl</TITLE></HEAD>
      <BODY>
       <CENTER><H1>Listado de Alumnos</H1></CENTER>
        <DIV STYLE="TOP:10;LEFT:100;POSITION:RELATIVE">
          <xsl:for-each select="notas/alumno">
              <xsl:sort select=" apellidos " order="ascending "/>
            <UL>
              <LI>
                <xsl:value-of select="apellidos"/>,
                <xsl:value-of select="nombre"/>
                (<xsl:value-of select="nexped"/>)
              </LI>
            </UL>
            <xsl:comment>
              Las notas obtenidas:
              <xsl:value-of select="nota-teoria"/>,
              <xsl:value-of select="nota-practica"/>-
              Nota final:<xsl:value-of select="nota-final"/>
            </xsl:comment>
          </xsl:for-each>
        </DIV>
      </BODY>
    </HTML>
  </xsl:template>

</xsl:stylesheet>
```

Aprovechando este último código, que contiene la sentencia de ordenación **<xsl:sort>** asociada a **<xsl:for-each>**, hay que hacer notar nuevamente que su utilización tiene una connotación diferente al utilizarse con **<xsl:template match=””>**.

En el siguiente código puede observarse cómo la sentencia <**xsl:sort**> pierde su formato de autocierre para englobar aquel fragmento de datos que queda afectado por dicha ordenación:

```
<?xml version="1.0" encoding="ISO-8859-1" ?>
<xsl:stylesheet version="1.0" xmlns:xsl="http://www.w3.org/1999/XSL/Transform">
 <xsl:template match="/">
  <HTML>
   <HEAD><TITLE>Ejemplo9b.xsl</TITLE></HEAD>
   <BODY>
    <CENTER><H1>Listado de Alumnos</H1></CENTER>
     <DIV STYLE="TOP:10;LEFT:100;POSITION:RELATIVE">

    </DIV>
   </BODY>
  </HTML>
 </xsl:template>

<xsl:template match="/notas/alumno">
   <xsl:sort select=" apellidos " order="ascending ">
          <UL>
           <LI>
            <xsl:value-of select="apellidos"/>,
            <xsl:value-of select="nombre"/>
            (<xsl:value-of select="nexped"/>)
           </LI>
          </UL>
          <xsl:comment>
           Las notas obtenidas:
           <xsl:value-of select="nota-teoria"/>,
           <xsl:value-of select="nota-practica"/>-
           Nota final:<xsl:value-of select="nota-final"/>
          </xsl:comment>
   </xsl:sort>
 </xsl:template>

</xsl:stylesheet>
```

Nota: a la hora de insertar el namespace de la hoja de estilo en el código XSL, el que la W3C recomienda es el que aparece en todos los ejemplos. ¡Cuidado!, porque puede suceder que investigando por Internet o libros existan ejemplos (patrocinados por Microsoft) que incluyan un namespace como <xsl:stylesheet version="1.0" **xmlns:xsl="http://www.w3.org/TR/WD-xsl"**> que al ser utilizados en nuestros códigos impidan su correcto funcionamiento.